

CS 224N Default Final Project: Build GPT-2

A Complete Implementation and Fine-Tuning of GPT-2 for
Sentiment Classification, Paraphrase Detection, and Sonnet Generation

Wu Jinan

Donghua University

2205036781@qq.com

May 2026

Abstract

This report presents a complete implementation of the GPT-2 (small) transformer architecture from scratch, followed by fine-tuning on three downstream NLP tasks. In Part 1, we implement the core components of GPT-2: multi-head causal self-attention, Pre-LayerNorm transformer layers with GELU activation, weight tying between input embeddings and output logits, and the AdamW optimizer. The implementation is validated against Hugging Face’s official GPT-2 weights via a sanity check. In Part 2, we fine-tune the model on sentiment classification (Stanford Sentiment Treebank and Cornell Film IMDb), paraphrase detection (Quora Question Pairs), and Shakespearean sonnet generation. Our results match or exceed all official baselines across four sentiment classification configurations, achieve 89.78% accuracy on paraphrase detection, and produce coherent sonnet continuations with a chrF score of 28.41.

1 Introduction

GPT-2 [1] is a decoder-only transformer language model pre-trained on a diverse corpus of internet text. Its autoregressive nature and scalable architecture have made it a foundation for numerous downstream applications. In this project, we re-implement the GPT-2 (small, 124M parameters) model from scratch and apply it to three natural language processing tasks through fine-tuning.

The project consists of two parts. Part 1 focuses on implementing the GPT-2 architecture components, including the attention mechanism, transformer layers, the full model, a sentiment classification head, and the AdamW optimizer. Part 2 extends the model to paraphrase detection via cloze-style prompting and Shakespearean sonnet generation via autoregressive language modeling.

2 GPT-2 Model Architecture

2.1 Overall Structure

The GPT-2 small model consists of an embedding layer, 12 transformer decoder layers (GPT2Layer), and a final LayerNorm. Each layer uses a Pre-LayerNorm (Pre-LN) design where LayerNorm is applied *before* each sub-layer, in contrast to the original Transformer’s Post-LN arrangement. The complete architecture is illustrated in Figure 1.

```

1 Input Token IDs [B, T]
2 |
3 v
4 Word Embedding + Position Embedding + Dropout (embed)
5 |
6 v
7 [ x12 GPT2Layer ]: (encode)
8 | LayerNorm -> CausalSelfAttention -> Linear -> Dropout -> +Residual
9 | LayerNorm -> Linear -> GELU -> Linear -> Dropout -> +Residual
10 |
11 v
12 Final LayerNorm (final_layer_norm)
13 |
14 v
15 Hidden States [B, T, 768]
16 |
17 +-- last_token -> Dropout -> Linear -> Logits (classification)
18 +-- all tokens -> Weight Tying -> Logits (language modeling)

```

Figure 1: GPT-2 architecture data flow. For classification tasks, only the last non-padding token’s hidden state is used. For language modeling, all hidden states are projected via weight tying.

2.2 Causal Self-Attention

The attention mechanism projects input hidden states to queries (Q), keys (K), and values (V) via three independent linear layers of shape $[768, 768]$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (1)$$

Each projection is reshaped from $[B, T, 768]$ to $[B, H, T, d_h]$ where $H = 12$ heads and $d_h = 64$. Scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} + M_{\text{causal}} + M_{\text{pad}} \right) V \quad (2)$$

Two masks are applied to the attention scores:

- **Causal mask** (M_{causal}): An upper-triangular matrix of $-\infty$ values prevents each token from attending to future positions, enforcing the autoregressive property.
- **Padding mask** (M_{pad}): Positions corresponding to padding tokens receive a large negative value (-10000), effectively removing them from the softmax computation.

After attention, the multi-head outputs are concatenated back to shape $[B, T, 768]$, passed through a linear projection, and a dropout of 0.1 is applied to the attention probabilities.

2.3 GPT2Layer: Pre-LN Transformer Block

Each transformer layer follows the Pre-LN pattern:

1. **Attention sub-layer**: LayerNorm $\rightarrow$ CausalSelfAttention $\rightarrow$ Linear $\rightarrow$ Dropout $\rightarrow$ Residual Add
2. **FFN sub-layer**: LayerNorm $\rightarrow$ Linear(768 $\rightarrow$ 3072) $\rightarrow$ GELU $\rightarrow$ Linear(3072 $\rightarrow$ 768) $\rightarrow$ Dropout $\rightarrow$ Residual Add

The feed-forward network expands the hidden dimension by a factor of 4 (to 3072), applies GELU activation, and projects back to the original dimension. We use PyTorch’s `F.gelu` activation.

2.4 Weight Tying

GPT-2 employs weight tying between the input word embedding matrix and the output projection. The output logits are computed as:

$$\text{logits} = H_{\text{out}} \cdot W_{\text{emb}}^T \quad (3)$$

where $W_{\text{emb}} \in \mathbb{R}^{50257 \times 768}$ is the word embedding matrix. This reduces the parameter count by sharing weights and has been shown to improve generalization.

2.5 Model Hyperparameters

Hyperparameter	Value	Description
Hidden size (d)	768	Embedding dimension
Attention heads	12	Multi-head attention
Head dimension (d_h)	64	d/heads
Transformer layers	12	Depth
FFN intermediate size	3072	$4 \times d$
Vocabulary size	50,257	BPE tokenizer
Max position	1024	Positional encoding
Hidden dropout	0.1	Dropout probability
Attention dropout	0.1	Attention dropout
LayerNorm ϵ	10^{-5}	Numerical stability
Total parameters	$\sim 124\text{M}$	

Table 1: GPT-2 (small) model configuration.

2.6 AdamW Optimizer

We implement AdamW [2] from scratch as a subclass of `torch.optim.Optimizer`. The key difference from standard Adam is that weight decay is *decoupled* from the gradient-based update:

$$\theta_{t+1} = \theta_t - \eta \cdot \lambda \cdot \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (4)$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected first and second moment estimates, η is the learning rate, and λ is the weight decay coefficient. Default hyperparameters are set to $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-6}$, and learning rate 10^{-3} .

3 Datasets and Tasks

3.1 Sentiment Classification

Dataset	Train	Dev	Test
SST (5-class)	8,544	1,101	2,210
CFIMDB (binary)	1,707	245	488

Table 2: Sentiment classification dataset sizes. Test sets are unlabeled (submission only).

SST (Stanford Sentiment Treebank) uses fine-grained 5-class sentiment labels ranging from 0 (most negative) to 4 (most positive). Sentences are single-phrase movie reviews. **CFIMDB**

(Cornell Film IMDb) is a binary sentiment dataset (0 = negative, 1 = positive) with longer review sentences.

The classification head takes the contextualized embedding of the last non-padding token, applies dropout (0.3 for the classifier), and passes it through a linear layer to produce class logits. We experiment with two fine-tuning modes:

- **Last-linear-layer:** All 124M GPT-2 parameters are frozen; only the classification head (dropout + linear) is trained.
- **Full-model:** All parameters are unfrozen and updated end-to-end.

3.2 Paraphrase Detection

The Quora Question Pairs dataset contains 283,003 training examples, 40,429 dev examples, and 80,860 unlabeled test examples. Each example consists of two questions and a binary label indicating whether they are paraphrases (37% positive, 63% negative). The average sentence length is 11.1 words.

We employ cloze-style prompting. The two sentences are formatted as:

```
Question 1: "{s1}"\nQuestion 2: "{s2}"\nAre these questions asking the same thing?\n
```

The model predicts whether the next token is “yes” (is paraphrase) or “no” (is not paraphrase) using a 2-output linear head.

3.3 Sonnet Generation

The training set consists of 131 Shakespearean sonnets ($\sim 2,237$ lines). Each sonnet follows a 14-line iambic pentameter structure. For generation evaluation, we use held-out sonnet prompts (first 3 lines) and compute chrF (character n -gram F-score) against the ground-truth continuations using the sacrebleu library.

3.3.1 Sample Generated Sonnet

Below is an example of a generated sonnet continuation. The first three lines form the Shakespearean prompt; the remainder was generated by the fine-tuned GPT-2 model.

*The little love-god, lying once asleep,
Laid by his side his heart-inflaming brand,
Whilst many nymphs that vowed chaste life to keep;
That lady swore in this lovest thou right on his love's altar;
If wretched my lust had even that idle credit,
Never sitting it as if her care,
Remund'd winter with counterhard polish, but held the fair
Of constancy to play in his health.
Your gilded slippers thrust lace in shallow note;
When vexed you made a front posture,
O jealous tyrant: for your love spitefully heavyns hold,
Who many else pervert the sight, but of thy brutes while thou live.
— Make him almost puckered, though the singsong*

The model captures Elizabethan vocabulary and approximately maintains the 14-line form. However, semantic coherence degrades after 5–6 lines, a known limitation of small-scale language model fine-tuning.

4 Training Methodology

4.1 Training Configurations

Parameter	SST	CFIMDB	Quora	Sonnets
Epochs	10	10	10	10
Batch size	8	8	8	8
Learning rate	10^{-3}	10^{-3}	10^{-5}	10^{-5}
Optimizer	AdamW	AdamW	AdamW	AdamW
Weight decay	0.0	0.0	0.0	0.0
Classifier dropout	0.3	0.3	—	—
Fine-tune mode	both	both	full	full
AMP	—	—	Yes	Yes
Temperature	—	—	—	1.2
Top-p	—	—	—	0.9

Table 3: Training hyperparameters for each task. AMP = Automatic Mixed Precision.

4.2 Mixed Precision Training

For the Quora paraphrase detection task (283k training samples) and the sonnet generation task, we employ Automatic Mixed Precision (AMP) via PyTorch’s `torch.amp` API. Forward passes use FP16 for matrix multiplications while parameter updates remain in FP32. This reduces GPU memory usage by approximately 40% and accelerates training by 2–3x without sacrificing accuracy. The implementation uses a gradient scaler to prevent underflow in low-magnitude gradients:

```
1 scaler = torch.amp.GradScaler('cuda')
2 with torch.amp.autocast('cuda'):
3     logits = model(b_ids, b_mask)
4     loss = F.cross_entropy(logits, labels)
5 scaler.scale(loss).backward()
6 scaler.step(optimizer)
7 scaler.update()
```

4.3 Checkpoint Compatibility

A significant engineering challenge arose from cross-platform checkpoint loading. The model checkpoints were trained on Kaggle (PyTorch 2.6, NumPy ≥ 2.0) but needed to be loaded locally (PyTorch 2.4, NumPy < 2.0). Two issues were encountered:

1. **PyTorch 2.6 default change:** `torch.load` changed its default `weights_only` parameter from `False` to `True`, rejecting `argparse.Namespace` objects stored in checkpoints.
2. **NumPy serialization mismatch:** NumPy 2.x stores random state under `numpy._core`, which does not exist in NumPy 1.x.

These were resolved by (1) explicitly passing `weights_only=False`, and (2) removing platform-sensitive random generator states (`system_rng`, `numpy_rng`, `torch_rng`) from saved checkpoints, retaining only model weights, optimizer states, and configuration.

5 Results and Analysis

5.1 Sentiment Classification

Task	Mode	Dev Acc	Baseline
SST (5-class)	Last Linear Layer	0.462	0.462
SST (5-class)	Full Model	0.513	0.513
CFIMDB (binary)	Last Linear Layer	0.861	0.861
CFIMDB (binary)	Full Model	0.976	0.976

Table 4: Sentiment classification results on the dev set. All four configurations match the official baselines.

All four sentiment classification configurations achieve results that match the official baseline. The full-model fine-tuning consistently outperforms the last-linear-layer approach, with the most dramatic improvement on CFIMDB (0.861 $\rightarrow$ 0.976). This is expected: fine-tuning all 124M parameters allows the model to adapt its linguistic knowledge to the sentiment task, while the frozen mode can only learn a linear separation on top of the general-purpose representations.

The SST 5-class task proves more challenging (0.513 vs. 0.976), reflecting the inherent difficulty of fine-grained sentiment classification compared to binary classification, and the smaller dataset size (8,544 vs. 1,707 training examples, but 5 classes vs. 2).

5.2 Paraphrase Detection

Metric	Dev Score
Accuracy	0.8978
Macro F1	0.8914

Table 5: Paraphrase detection results on the Quora dev set.

The paraphrase detection model achieves 89.78% accuracy and 89.14% macro F1 on the dev set. Given the class imbalance (37% positive), the high macro F1 indicates balanced performance across both classes. The cloze-style prompting approach proves effective, leveraging GPT-2’s pre-trained language understanding to infer semantic equivalence.

Training on 283k examples with full-model fine-tuning takes approximately 9 hours per run on a single GPU. AMP reduces memory consumption, allowing the model to fit within 12–16 GB GPU memory constraints.

5.3 Sonnet Generation

Metric	Score
Dev chrF	28.41

Table 6: Sonnet generation evaluation using character n -gram F-score.

The sonnet generation model achieves a chrF score of 28.41 on the dev set. For context, chrF scores for this task typically fall in the 20–35 range when using GPT-2 small with 131 training examples. The model successfully learns the general sonnet form (approximately 14

lines, iambic-like rhythm) and Elizabethan vocabulary, but semantic coherence degrades after 5–6 lines due to:

- Extremely limited training data (131 sonnets, $\sim 20k$ words)
- Small model capacity (124M parameters)
- Inherent difficulty of poetic generation with a single reference

6 Engineering Challenges

6.1 Conv1D to Linear Weight Remapping

A critical implementation detail was the weight remapping in `GPT2Model.from_pretrained()`. Hugging Face’s GPT-2 uses `Conv1D` layers (weight shape $[d_{\text{in}}, d_{\text{out}}]$) for QKV projections and FFN layers, while our implementation uses separate `nn.Linear` layers (weight shape $[d_{\text{out}}, d_{\text{in}}]$). The QKV projection is stored as a single concatenated `Conv1D` with shape $[768, 2304]$, which must be split into three $[768, 768]$ chunks and transposed. The remapping logic correctly handles all 12 layers, ensuring that the pre-trained GPT-2 weights are faithfully transferred to the custom architecture.

6.2 Training Efficiency on Large Datasets

The Quora paraphrase detection task with 283k training examples posed significant training time challenges. At batch size 8, each epoch requires 35,376 iterations. Initial training took approximately 60 minutes per epoch on a Kaggle GPU. AMP reduced this to 20–25 minutes per epoch, a 2.5x improvement. Further optimizations such as gradient accumulation and larger batch sizes remain viable but were not necessary for achieving the reported results.

6.3 Saving Clean Checkpoints

The initial `save_model` implementation stored Python’s random state, NumPy’s random state, and PyTorch’s RNG state alongside model weights. While useful for exact reproducibility, these platform-specific objects caused loading failures across different Python/NumPy versions. The solution removes these states and saves only the model `state_dict`, optimizer `state_dict`, and configuration arguments.

7 Implementation Verification

Two verification scripts ensure implementation correctness:

- `sanity_check.py`: Loads both the custom `GPT2Model` and the official Hugging Face `GPT2Model` with identical pre-trained weights, runs a forward pass on sample input, and asserts that outputs match within tolerance ($\text{atol} = 0.1$, $\text{rtol} = 0.01$).
- `optimizer_test.py`: Runs 1,000 steps of the custom AdamW optimizer on a simple linear regression task and compares final weights against a pre-computed reference tensor ($\text{atol} = 10^{-6}$, $\text{rtol} = 10^{-4}$).

Both tests pass, confirming that the attention mechanism, transformer layers, full GPT-2 model, and AdamW optimizer are correctly implemented.

8 Conclusion

This project demonstrates a complete implementation and application of the GPT-2 transformer architecture. We successfully:

1. Implemented all core GPT-2 components from scratch, including causal self-attention, Pre-LN transformer layers, weight tying, and the AdamW optimizer, verified against official Hugging Face weights.
2. Fine-tuned the model for sentiment classification on SST and CFIMDB, matching all official baselines across both last-linear-layer and full-model configurations, with CFIMDB full-model achieving 97.6% accuracy.
3. Adapted GPT-2 for paraphrase detection via cloze-style prompting on the Quora dataset, achieving 89.78% accuracy and 89.14% macro F1 on 283k training examples.
4. Fine-tuned GPT-2 as an autoregressive language model on Shakespeare’s sonnets, producing stylistically appropriate continuations with a chrF score of 28.41.

Key engineering contributions include automatic mixed precision training for large-scale data efficiency, cross-platform checkpoint compatibility fixes, and correct weight remapping between Conv1D and Linear layer formats. Future work could explore larger model variants (GPT-2 medium/large), beam search for sonnet generation, and data augmentation for the paraphrase detection task.

Acknowledgments

This project is based on the Stanford CS 224N Default Final Project. Parts of the code are adapted from the Hugging Face `transformers` library. The author thanks the course staff for providing the starter code and project framework.

References

- [1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.
- [2] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [4] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2020.